ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ МОЛДОВЫ

ФАКУЛЬТЕТ МАТЕМАТИКИ И ИНФОРМАТИКИ ДЕПАРТАМЕНТ ИНФОРМАТИКИ

Дулева Анастасия IA2503

Отчет

по дисциплине „PYTHON для приложений”

Руководитель: _____ Плешка Наталья, лектор

Кишинев, 2026

1. Краткая постановка задачи 2
2. Логика реализованных алгоритмов 2
2.1. Основной алгоритм программы 2
2.2. Алгоритм добавления преподавателя 2
2.3. Алгоритм добавления дисциплин 2
2.4. Алгоритм изменения дисциплины 3
2.5 Алгоритм отображения списка 3
2.6 Алгоритм удаления преподавателя 3
2.7 Алгоритм подсчета статистики 3
2.8 Алгоритм поиска максимума 4
2.9 Алгоритм поиска по имени 4
2.10 Алгоритм уникальных дисциплин 4
2.11 Алгоритм обработки ошибок 4
3. Скриншоты интерфейса 4
4. Структуры данных 7
Различия между коллекциями данных и файлами данных 8
Код загрузки из файла 8
Код сохранения в файл 9
5. Описание функционала с примерами кода 9
6. Выводы 14
Заключение: 14

1. Краткая постановка задачи

Цель работы: Разобрать программу на Python для управления данными о преподавателях и читаемых ими дисциплинах.

Требования к программе: Хранение данных в словаре; сохранение данных в текстовой файл; проверка ввода с помощью регулярных выражений; меню с 10 функциями; обработка ошибок ввода; модульная структура.

2. Логика реализованных алгоритмов

2.1. Основной алгоритм программы

1. Загрузка данных из файла в словарь при запуске
2. Бесконечный цикл с отображением меню
3. Обработка выбора пользователя вызовом соответствующей функции
4. Автоматическое сохранение изменений в файл
5. Выход из цикла при выборе пункта 10

2.2. Алгоритм добавления преподавателя

1. Ввод имени с проверкой через регулярное выражение
2. Ввод фамилии с такой же проверкой
3. Формирование ключа “имя_фамилия”
4. Проверка уникальности через lower()
5. Добавление в словарь с пустым списком дисциплин
6. Запрос на немедленное добавление дисциплин
7. Сохранение в файл

2.3. Алгоритм добавления дисциплин

1. Проверка существования преподавателя
2. Цикл ввода дисциплин:
   - Ввод названия с проверкой через regex
   - Проверка на дубликат дисциплины у данного преподавателя
   - Добавление в список, если дисциплина новая
3. Запрос “Добавить еще?” после каждой дисциплины
4. Выходит из цикла при ответе не равном “Да”
5. Сохранение в файл

2.4. Алгоритм изменения дисциплины

1. Вывод списка преподавателей
2. Выбор преподавателя по имени_фамилии
3. Вывод списка его дисциплин
4. Выбор дисциплины для изменения
5. Ввод нового названия с проверкой через regex

6. Замена старого названия на новое в списке
7. Сохранение в файл

2.5 Алгоритм отображения списка

1. Проверка на пустой словарь
2. Для каждого преподавателя:
   - Вывод имени_фамилии
   - Вывод количества дисциплин
   - Построчный вывод всех дисциплин

2.6 Алгоритм удаления преподавателя

1. Вывод списка преподавателей
2. Ввод имени_фамилии для удаления
3. Запрос подтверждения (Y/N)
4. При подтверждении удаление из словаря (del)
5. Сохранение в файл

2.7 Алгоритм подсчета статистики

1. Общее число преподавателей = len(teachers)
2. Общее число дисциплин = сумма длин всех списков
3. Вывод результатов

2.8 Алгоритм поиска максимума

1. Поиск максимальной длины списка дисциплин
2. Если максимум = 0 сообщение “нет дисциплин”
3. Вывод всех преподавателей с длинной = максимум

2.9 Алгоритм поиска по имени

1. Ввод строки для поиска
2. Приведение к lower()

3. Поиск частичного совпадения в ключах (lower())

4. Вывод всех найденных преподавателей и их дисциплин

2.10 Алгоритм уникальных дисциплин

1. Создание пустого множества (set)

2. Добавление всех дисциплин во множество

3. Сортировка множества в алфавитном порядке

4. Построчный вывод

5. Вывод общего количества

2.11 Алгоритм обработки ошибок

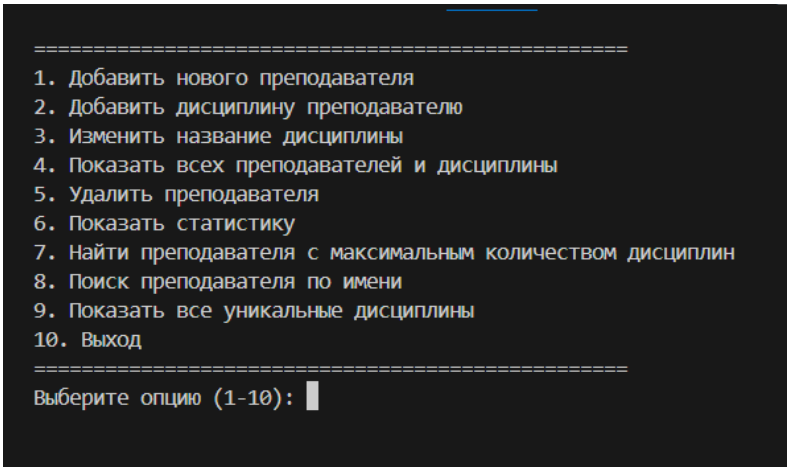
1. Блок try-except для перехвата исключений

2. Вывод сообщения об ошибке

3. Возврат к меню без завершения программы

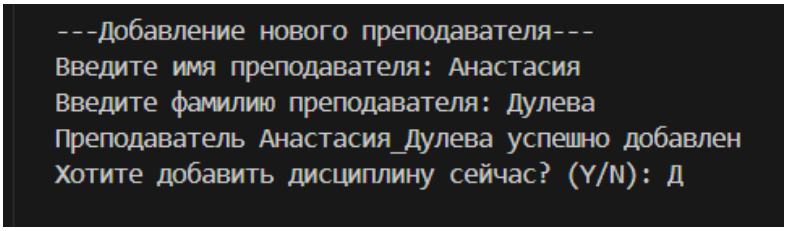
3. Скриншоты интерфейса

1. Главное меню программы



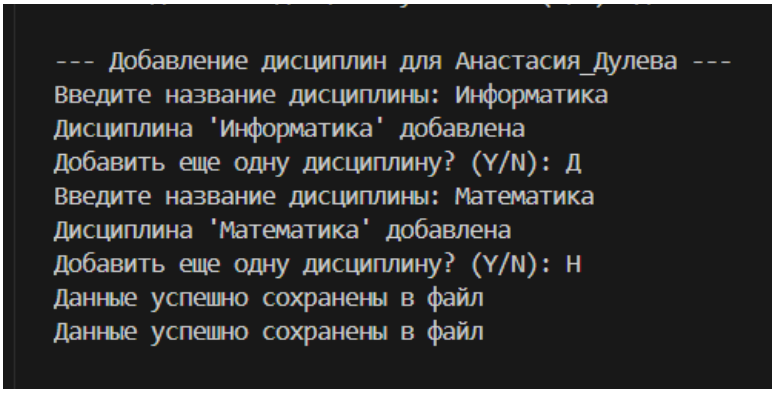
Главное меню предоставляет пользователю доступ ко всем функциям программы.

2. Добавление нового преподавателя



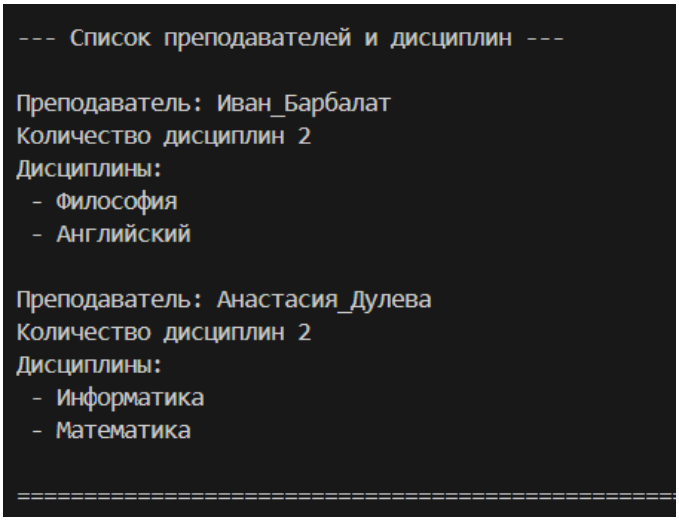
Демонстрирует процесс добавления преподавателя через дефис

3.Добавление дисциплин



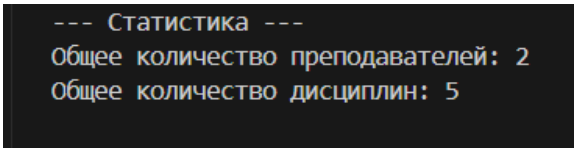
Показывает возможность добавления нескольких дисциплин с проверкой через цикл.

4.Просмотр списка преподавателей



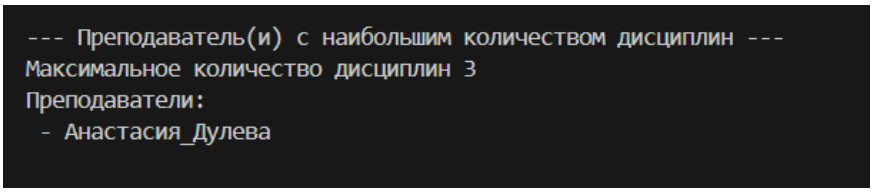
Отображает всех преподавателей с их дисциплинами и количеством дисциплин.

5.Статистика



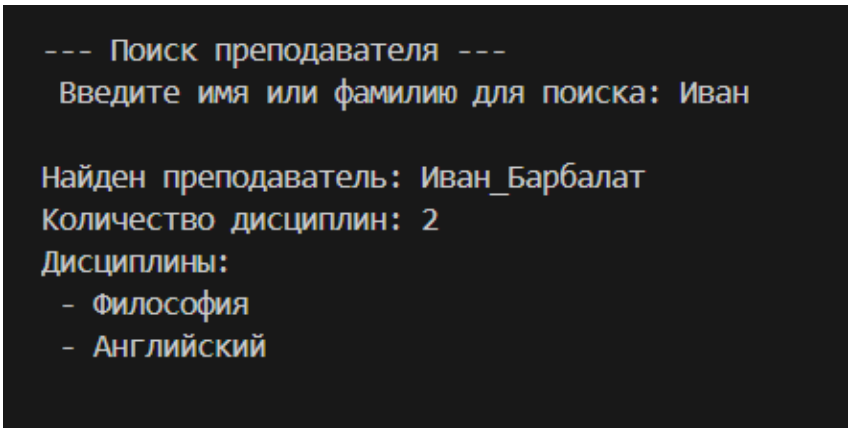
Определяет общее количество преподавателей и дисциплин.

6.Поиск преподавателя с макс.дисциплинами



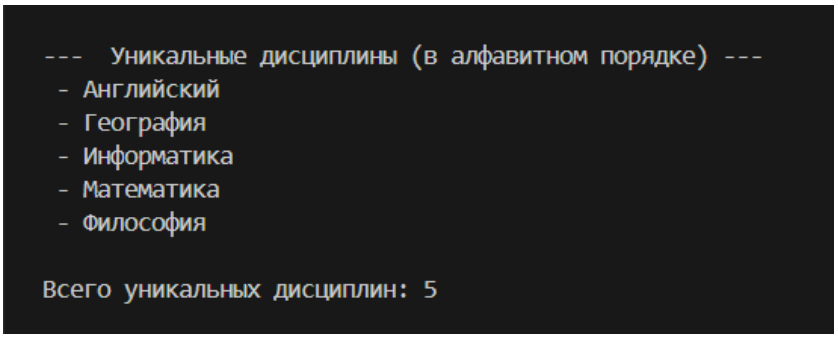
Поиск преподавателя с наибольшим количеством дисциплин.

7.Поиск по имени



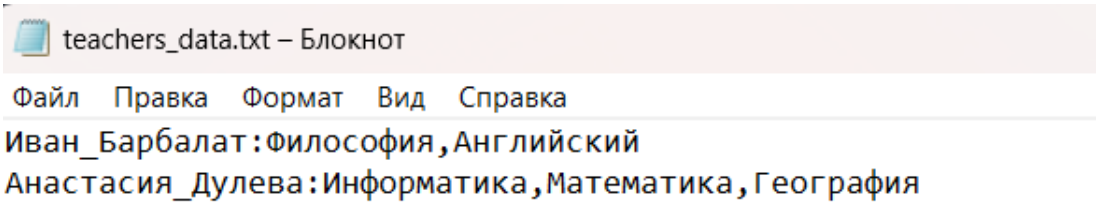
Поиск преподавателя по имени (частичное совпадение).

8.Уникальные дисциплины



Отображение всех уникальных дисциплин в алфавитном порядке.

9.Содержимое файла teachers_data.txt



Показывает, как данные сохраняются в файл для дальнейшего анализа.

4. Структуры данных

1. Словарь

```
teachers = tf.load_from_file(filename)
```

Преимущества использования словаря:

- Быстрый доступ по ключу
- Естественное отображение данных
- Автоматическая уникальность ключей
- Гибкость значений

2. Список

```
teachers[full_name] = []
```

Преимущества:

- Сохранение порядка
- Удобные методы
- Возможность изменять элементы
- Простой перебор

3. Множество

```
all_disciplines = set()
    for disciplines in teachers.values():
        all_disciplines.update(disciplines)
```

Преимущества:

- Автоматическая уникальность
- Быстрая проверка наличия
- Идеально для сбора уникальных значений

Различия между коллекциями данных и файлами данных

Характеристика	Коллекции в памяти (словарь, список)	Файл (teachers_data.txt)
Где хранится	В оперативной памяти	На жестком диске
Скорость доступа	Очень быстпый	Медленный
Сохранение после выхода	Нет	Да
Формат	Структурированные объекты Python	Текст (строки)
Размер	Ограничен объемом RAM	Ограничен местом на диске
Цель использования	Обработка данных во время работы программы	Долговременное хранение

Код загрузки из файла

```
def load_from_file(filename):
    teachers = {}
    if os.path.exists(filename):
        try:
            with open(filename, 'r', encoding='utf-8') as file:
                for line in file:
                    line = line.strip()
                    if line:
                        parts = line.split(':')
                        if len(parts) == 2:
                            name = parts[0]
                            disciplines = parts[1].split(',') if parts[1] else
[]
                            teachers[name] = disciplines
        except Exception as e:
            print(f"Ошибка при загрузке файла: {e}")
    return teachers
```

При загрузке программа читает текстовой файл и преобразует его в словарь для быстрой обработки.

Код сохранения в файл

```
def save_to_file(filename, teachers):
    try:
        with open(filename, 'w', encoding='utf-8') as file:
            for name, disciplines in teachers.items():
                disciplines_str = ','.join(disciplines)
```

```
                file.write(f"{name}:{disciplines_str}\n")
        print("Данные успешно сохранены в файл")
    except Exception as e:
        print(f"Ошибка при сохранении в файл: {e}")
```

После каждого изменения программа преобразует словарь обратно в текст и сохраняет на диск.

5. Описание функционала с примерами кода

1. Модульная структура программы

```
import teacher_functions as tf
```

Программа разделена на два файла согласно требованию задания. main.py отвечает за интерфейс и меню, а teacher_functions.py содержит все рабочие функции. Это делает код структурированным и легко поддерживаемым.

2. Проверка данных с помощью регулярных выражений

```
def validate_name(name):
    pattern = r'^[A-Za-zA-Яа-я]+(-[A-Za-zA-Яа-я]+)?$'
    if 1 <= len(name) <= 30 and re.match(pattern, name):
        return True
    return False

def validate_discipline(discipline):
    pattern = r'^[A-Za-zA-Яа-я0-9\s]{2,50}$'
    if re.match(pattern, discipline):
       return True
    return False
```

Регулярные выражения обеспечивают строгую проверку вводимых данных:

Для имени/фамилии: только буквы, возможен дефис для составных имен;

Для дисциплины: буквы, цифры и пробелы, длина от 2 до 50 символов.

3. Добавление нового преподавателя

```
def add_teacher(teachers, filename):
    print("\n---Добавление нового преподавателя---")

    while True:
        first_name = input("Введите имя преподавателя: ").strip()
```

```
        if validate_name(first_name):
            break
        print("Ошибка! Имя должно содержать только буквы (1-30 символов),
может содержать дефис")

    while  True:
        last_name = input("Введите фамилию преподавателя:").strip()
        if validate_name(last_name):
            break
        print("Ошибка! Фамилия должно содержать только буквы (1-30 символов),
может содержать дефис")

    full_name = f"{first_name}_{last_name}"
name_exists = False
    for existing_name in teachers:
        if existing_name.lower() == full_name.lower():
            name_exists = True
            break

    if name_exists:
        print(f"Преподаватель {full_name} уже существует!")
        return

    teachers[full_name] = []
    print(f"Преподаватель {full_name} успешно добавлен")
```

Функция реализует:

Циклы для повторного ввода при ошибке (пока пользователь не введет корректные данные); Проверку через .lower() для регистронезависимого сравнения; Добавление в словарь с пустым списком дисциплин.

4. Добавление нескольких дисциплин

```
def add_disciplines_to_teacher(teachers, teacher_name, filename):
    if teacher_name not in teachers:
        print(f"Преподаватель {teacher_name} не найден!")
        return

    print(f"\n--- Добавление дисциплин для {teacher_name} ---")

    while True:
        while True:
            discipline = input("Введите название дисциплины: ").strip()
            if validate_discipline(discipline):
                break
```

```
            print("Ошибка! Дисциплина должна содержать буквы, цифры, пробелы
(2-50 символов)")

        if discipline not in teachers[teacher_name]:
            teachers[teacher_name].append(discipline)
            print(f"Дисциплина '{discipline}' добавлена")
        else:
            print(f"Дисциплина '{discipline}' уже есть у преподавателя")

        choice = input("Добавить еще одну дисциплину? (Y/N): ").upper()
        if choice != 'Y' and choice != 'Д':
            break
```

Реализовано требование пункта 2а:

- Внутренний цикл для проверки каждой дисциплины
- Внешний цикл для возможности добавления нескольких дисциплин
- Проверка ответа через .upper() для поддержки Y/Д

5. Отображение списка с количеством дисциплин

```
def display_teachers(teachers):
    print("\n--- Список преподавателей и дисциплин ---")

    if not teachers:
        print("Список преподавателей пуст!")
        return

    for name, disciplines in teachers.items():
        print(f"\nПреподаватель: {name}")
        print(f"Количество дисциплин {len(disciplines)}")
        if disciplines:
            print("Дисциплины:")
            for disc in disciplines:
                print(f" - {disc}")
        else:
            print(" Нет дисциплин")
```

Функция выводит:

Количество дисциплин для каждого преподавателя; список всех дисциплин с отступами.

6. Подсчет статистики

```
def get_statistics(teachers):
    print("\n--- Статистика ---")
```

```
    total_teachers = len(teachers)

    total_disciplines = 0
    for disciplines in teachers.values():
        total_disciplines += len(disciplines)

    print(f"Общее количество преподавателей: {total_teachers}")
    print(f"Общее количество дисциплин: {total_disciplines}")

    return total_teachers, total_disciplines
```

Функция работает напрямую с данными из файла (через словарь teachers):

len(teachers) дает общее количество преподавателей; сумма длин всех списков дает общее количество дисциплин.

7. Поиск преподавателей с максимальным количеством дисциплин

```
def find_teacher_with_max_disciplines(teachers):
    print("\n--- Преподаватель(и) с наибольшим количеством дисциплин ---")

    if not teachers:
        print("Список преподавателей пуст!")
        return

    max_disciplines = 0
    for disciplines in teachers.values():
        if len(disciplines) > max_disciplines:
            max_disciplines = len(disciplines)

    if max_disciplines == 0:
        print("У всех преподавателей нет дисциплин")
        return

    print(f"Максимальное количество дисциплин {max_disciplines}")
    print("Преподаватели:")
    for name, disciplines in teachers.items():
        if len(disciplines) == max_disciplines:
            print(f" - {name}")
```

Алгоритм сначала находит максимум, затем всех преподавателей, у которых количество дисциплин равно этому максимуму.

8. Уникальные дисциплины в алфавитном порядке

```
def display_unique_disciplines(teachers):
```

```
    print("\n---  Уникальные дисциплины (в алфавитном порядке) ---")

    all_disciplines = set()
    for disciplines in teachers.values():
        all_disciplines.update(disciplines)

    if not all_disciplines:
        print("Нет дисциплин")
        return

    sorted_disciplines = sorted(all_disciplines)
    for disc in sorted_disciplines:
        print(f" - {disc}")

    print(f"\nВсего уникальных дисциплин: {len(sorted_disciplines)}")
```

Множество (set) автоматически отсеивает дубликаты; функция sorted() сортирует в алфавитном порядке; выводится как список дисциплин, так и их общее количество.

9. Обработка ошибок

```
try:
            choice = input("Выберите опцию (1-10): ").strip()
except Exception as e:
            print(f"Произошла ошибка: {e}")
            print("Попробуйте снова")
```

Блок try-except перехватывает все возможные ошибки, благодоря чему программа выводит сообщение и возвращается к меню.

6. Выводы

В ходе выполнения данной работы я разработала программу на Python для управления данными о преподавателях и их дисциплинах.Основываясь на своем опыте, могу сделать следующие выводы:

Положительные стороны:

1. Простота и читаемость кода
2. Мощные встроенные структуры данных
3. Эффективная работа с файлами
4. Регулярные выражения
5. Обработка исключений

Сложности с которыми столкнулась:

1. Кодировка в Windows

2. Регулярные выражения для кириллицы

Заключение:

Python показал себя как отличный язык для решения поставленной задачи. Благодоря простому синтаксису и богатым встроенным возможностям, программа получилась компактной, но при этом полностью соответствующей всем требованиям задания. Большим плюсом оказалась возможность быстро реализовать логику с помощью встроенных структур данных (словари, списки, множества) и легкость работы с файлами для сохранения данных между запусками программы.

Несмотря на небольшой опыт изучения языка удалось создать полноценное приложение с меню, валидацией данных, работой с файлами и обработкой ошибок. Это подтверждает, что Python является отличным выбором для начинающих программистов.